



1. Introduzione



Lezione 1 - @September 24, 2025

Algoritmi

Procedimenti per la soluzione dei problemi

- sintesi di algoritmi: creo un algoritmo o cerco una soluzione che ha già fatto qualcun altro
- analisi di algoritmi: voglio capire quante risorse usa

Algoritmica

Disciplina dell'informatica che si occupa di tutti gli aspetti legati agli algoritmi

- progettazione
- strutture dati
- efficienza
- limitazioni e complessità
- nuovi modelli di calcolo

Strutture dati

Modi di organizzazione dei dati

Progettazione di Algoritmi

1. L'algoritmo è corretto? Risolve il problema per cui l'ho progettato
2. Efficienza: uso delle risorse
 - a. tempo
 - b. spazio
 - c. rete
 - d. processori
 - e. energia - meno anidride carbonica prodotta

Esempio del commesso viaggiatore

classico esempio di algoritmo inefficiente

Istanza - N città

Sono note le distanze tra le città.

Problema: trovare il percorso più breve che visiti tutte le città e torni al punto di partenza.

$$n(n-1)(n-2)\dots = n!$$

→ algoritmi approssimati, nella maggior parte dei casi va abbastanza bene

Perché studiare l'algoritmica?

aspetti pratici

- esigenza di risolvere problemi

aspetti teorici (circa)

| science is what we understand well enough to explain to a computer. art is everything else we do

Esempio 2

Fissato N

Dato elenco di $N - 1$ numeri distinti tra loro tra $1, N$ trovare quello che manca

→ **Formula di Gauss per la somma dei primi n numeri naturali** $S(n) = \frac{n(n+1)}{2}$.

Cos'è un algoritmo?



Insieme **ordinato e finito** di **passi eseguibili** e **non ambigui** che definiscono un **procedimento che termina**

La randomizzazione è utile per velocizzare i calcoli a volte

Affidandosi al caso è possibile trovare la risposta corretta.

Algoritmo e Programma

Un programma è un algoritmo scritto in un linguaggio di programmazione

Pseudocodice (algol-like)

```
ALGORITMO moltiplicazione(intero a, intero b) → intero
  RETURN a * b
```



Lezione 2 - @September 26, 2025

Algoritmo: insieme ordinato... ([Lezione 1 - Definizione](#))

Programma: insieme *ordinato e finito* di *istruzioni scritte* secondo le regole di uno specifico linguaggio di programmazione

Sintesi di algoritmi

→ Progettare algoritmi

Parto da un problema e voglio trovare un algoritmo risolutivo

Una volta progettato, devo capire:

- se il problema viene risolto
 - se è efficiente, quante risorse impiega (es. tempo, spazio)

Come fare l'analisi di un algoritmo?

valutazione a posteriori

- faccio "girare" il programma che implementa l'algoritmo - **testing**
 - funziona sempre? non posso testare infiniti casi
 - quanto ci mette?
 - Diverse soluzioni, qual è la migliore?

La codifica dell'algoritmo richiede molto tempo

valutazione a priori

- analizzo l'algoritmo in **fase di progettazione**, evitando la codifica

Vari modi di costruire l'algoritmo della moltiplicazione

$a, b > 0$

$19 \times$
 $114 =$
 2166

```
#prodotto su 1 cifra = #cifre(a) x #cifre(b)
#somme = #cifre(b) somme a più cifra
```

In generale, il numero di cifre per esprimere il base il numero b è $\log 2$

Somme iterate

$$a \times b = a + a + \dots + a$$

b volte

```
ALGORITMO moltiplicazione(intero a, intero b) → intero
  prod ← 0
  WHILE b > 0 DO
    prod ← prod + a
    b ← b - 1
  RETURN prod
```

analizzo l'esecuzione

$$b_i = b - i$$
$$prod_i = a \times i$$

Quando **WHILE** termina

$$b_i = 0$$
$$i = b$$
$$prod_b = a \times b$$

Quanto dura l'esecuzione?

- suppongo che ogni riga di codice costa tempo 1 → 5 righe di codice nel programma
- linee 1, 5 vengono eseguite 1 volta
- linee 3, 4 vengono eseguite b volte
- linea 2 viene eseguita $b + 1$ volte

Tempo
2
$2b$
$b + 1$
$= 3b + 3$

$$\rightarrow T(a, b) = 3b + 3$$

Nuovo algoritmo per la moltiplicazione



Questo algoritmo è comunemente usato nelle CPU, al posto di moltiplicare e dividere per due si shiftano i bit a dx o sx

a	b	tutte le volte che ho un numero dispari, lo copio
moltiplicando $\times 2$	moltiplicatore $\times 2$	
19	114	
38	57	38
76	28	
152	14	
304	7	304
608	3	608
1216	1	1216
		$= 2166$

Non servono più le tabelline, in binario è molto efficiente → sono semplici shift!

Regola per numeri Reali

$$a \times b = 2a \times \frac{b}{2}$$

Per numeri interi

$$a \times b = \begin{cases} 2a \cdot \frac{b}{2} & \text{se } b \text{ è pari} \\ 2a \cdot \frac{b-1}{2} & \text{se } b \text{ è dispari} \end{cases}$$

ALGORITMO *moltiplicazione*(intero a, intero b) → intero

$$prod \leftarrow 0$$
WHILE $b > 0$ **DO**

IF b è *dispari* **THEN**

$$prod \leftarrow prod + a$$

$$b \leftarrow b/2 \quad // \text{ divisione intera}$$
$$a \leftarrow a * 2$$

RETURN $prod$

analisi esecuzione - dimostrazione correttezza

$$a_i \times b_i + prod_i = a \times b$$

dimostrazione per induzione



Proprietà:

$$a_i \cdot b_i + prod_i = a \cdot b$$

- base $i = 0$

$$\begin{aligned}a_0 &= a \\ b_0 &= b \\ prod_0 &= 0\end{aligned}$$

$$a_0 \times b_0 + prod_0 = a \times b + 0 = a \times b \text{ ✓}$$

- passo $i - 1 \implies i$
 - caso 1: b_{i-1} pari

$$\begin{aligned}prod_i &= prod_{i-1} \\ b_i &= \frac{b_{i-1}}{2} \\ a_i &= a_{i-1} \cdot 2\end{aligned}$$

$$\begin{aligned}a_i \cdot b_i + prod_i &= a_{i-1} \cdot 2 \cdot \frac{b_{i-1}}{2} + prod_{i-1} = a_{i-1} \cdot b_{i-1} + prod_{i-1} = a \cdot b \\ & \text{(=3) per ipotesi induttiva}\end{aligned}$$

- caso 2: b_{i-1} dispari

$$\begin{aligned}prod_i &= prod_{i-1} + a_{i-1} \\ b_i &= \frac{b_{i-1} - 1}{2} \\ a_i &= a_{i-1} \cdot 2\end{aligned}$$

$$\begin{aligned}a_i \cdot b_i + prod_i &= a_{i-1} \cdot 2 \cdot \frac{b_{i-1} - 1}{2} + prod_{i-1} + a_{i-1} = a_{i-1} \cdot b_{i-1} - a_{i-1} + prod_{i-1} + a_{i-1} = a_{i-1} \cdot b_{i-1} + prod_{i-1} = a \cdot b \\ & \text{(=4) per ipotesi induttiva}\end{aligned}$$

Quanto ci mette



$$u = \text{\#iterazioni ciclo WHILE}$$

- suppongo che ogni riga di codice costa tempo 1 $\rightarrow$ 7 righe di codice nel programma
- linee 1, 7 vengono eseguite 1 volta
- linee 3, 5, 6 vengono eseguite u volte
- linea 2 viene eseguita $u + 1$ volte
- linea 4 viene eseguita $\leq u$ volte

Tempo
2
$3u$
$u + 1$
$\leq u$
$\rightarrow 4u + 3 \leq T \leq 5u + 3$

$T(a, b) \leq 5u + 3$ worst case

b	u
0	0
1	1
2	2
3	2
4	3
5	3
6	3
8	4
...	...

analizzando l'andamento, si nota che per potenze di 2 il numero di cicli `while` cresce in maniera logaritmica, infatti si ha:
per $b > 0$

$$u = \lfloor \log_2(b) \rfloor + 1$$

$$T(a, b) \leq 5 \cdot (\lfloor \log_2(b) \rfloor + 1) + 3 = 5 \lfloor \log_2(b) \rfloor + 8$$

Algoritmi moltiplicazioni @Lezione 2

Somme iterate

funzione tempo:

$$T(a,b) = 3b + 3$$

spazio costante

Algoritmo “alla russa”



moltiplicazioni e divisioni per 2

funzione tempo

$$T(a,b) \leq 5\lfloor \log_2(b) \rfloor + 8$$

spazio costante

Grafico del comportamento delle funzioni

<https://www.desmos.com/calculator/rjtyysgd6d>

Il grafico verde esprime $f(x) = 3x + 3$, mentre quello viola $g(x) = 5\log_2 x + 8$

Analisi dell’andamento delle due funzioni verso l’infinito (per numeri molto grandi)

1 algoritmo somme iterate

2 algoritmo alla russa

b	1	2	...	8	...	1000	...	1’000’000
1	6	9		27		3003		3’000’003
2	8	13		23		53		103

Algoritmi per potenze

Siano:

$$x, y \text{ interi}, y \geq 0$$

Calcolare:

$$x^y = \underbrace{x \cdot x \cdot \dots \cdot x}_{y \text{ volte}}$$

1. prodotti interi

```
ALGORITMO potenza(intero  $x$ , intero  $y$ )  $\rightarrow$  intero
   $power \leftarrow 1$ 
  WHILE  $y > 0$  DO
     $power \leftarrow power * x$ 
     $y \leftarrow y - 1$ 
  RETURN  $power$ 
```

Se ogni riga di codice costa *tempo* 1:

$$T(x,y) = 3y + 3$$

possiamo fare di meglio?

2. Potenze ricorsive

- Se y è un numero reale

$$x^y = x^{\frac{y}{2} \cdot 2} = (x^{\frac{y}{2}})^2$$

- Se y è un numero intero (≥ 0)

$$x^y = \begin{cases} 1 & y = 0 \\ (x^{\frac{y}{2}})^2 & y > 0, \text{ pari} \\ (x^{\frac{y-1}{2}})^2 \cdot x & y > 0, \text{ dispari} \end{cases}$$

pseudo-codice:

```

ALGORITMO potenza(intero x, intero y) → intero
  IF y = 0 THEN
    RETURN 1
  ELSE
    power ← potenza(x, y/2)    // divisione intera
    power ← power * power
    IF y è dispari THEN
      power ← power * x
    RETURN power

```

Se ogni riga di codice costa 1, quanto tempo impiega il programma?

- se $y = 0$: linea 1,2 1 volta $\quad \quad \quad | \rightarrow T(x, y) = 2$
- se $y > 0$: linee 1, 3-5, 7, 1 volta $\quad \quad \quad | \rightarrow T = 5$

linea 2: 0 volte $\quad \quad \quad | \rightarrow T = 0$
 linea 6: ≥ 1 volta $\quad \quad \quad | \rightarrow T = 1$
 linea 3: **chiamata ricorsiva** $\quad \rightarrow T(x, \lfloor \frac{y}{2} \rfloor)$
 $\rightarrow$ L'algoritmo impiega, *sommando i tempi*, $6 + T(x, \lfloor \frac{y}{2} \rfloor)$

Funzione tempo espressa tramite **funzione di ricorrenza**

$$T(x, y) = \begin{cases} 2 & y = 0 \\ 6 + T(x, \frac{y}{2}) & \text{altrimenti} \end{cases}$$

Come cresce il tempo?

$$\begin{aligned} T(x, 0) &= 2 \\ T(x, 1) &= 6 + T(x, 0) = 6 + 2 = 8 \end{aligned}$$

supponendo y potenze di 2

$$\begin{aligned} T(x, y) &= 6 + T(x, \frac{y}{2}) = \\ &= 6 + 6 + T(x, \frac{y}{2^2}) \\ &= 6 + 6 + 6 + T(x, \frac{y}{2^3}) \\ &\quad \dots \\ &= 6k + T(x, \frac{y}{2^k}) \end{aligned}$$

$$\frac{y}{2^k} = 1 \text{ per } y = 2^k, k = \log_2 y$$

Sostituendo nell'equazione

$$\begin{aligned} &= 6\log_2 y + T(x, 1) \\ &= 6\log_2 y + 8 \end{aligned}$$

Quindi:

$$T(x, y) = 6 \cdot \log_2 y + 8$$

E lo spazio?

la ricorsione non è "gratis"

- record di attivazione

altezza della pila

$$H(x, y) = \begin{cases} 1 & y = 0 \\ 1 + H(x, \lfloor \frac{y}{2} \rfloor) & y > 0 \end{cases}$$

... calcoli...

$$H(x, y) = 2 + \log_2 y$$

Alcune considerazioni sull’algoritmo delle potenze

Nella prima versione, la funzione tempo è espressa come $T(x, y) = 3y + 3 = O(n)$

Con la seconda versione dell’algoritmo, il tempo è dato da $T(x, y) = 6 \cdot \log_2 y + 8 = O(\log n)$

Nuovo algoritmo per le potenze

```
ALGORITMO potenza(intero x, intero y) → intero
  IF y = 0 THEN
    RETURN 1
  ELSE
    power ← potenza(x, y/2) * potenza(x, y/2)
    IF y è dispari THEN
      power ← power * x
    RETURN power
```

Questo algoritmo risulterà più lento, per via della doppia chiamata a funzione ricorsiva

Funzione di ricorrenza per il nuovo algoritmo

$$T(x, y) = \begin{cases} 2 & y = 0 \\ 5 + 2 \cdot T(x, \lfloor \frac{y}{2} \rfloor) & \text{altrimenti} \end{cases}$$

Algoritmo equivalente

```
ALGORITMO potenza(intero x, intero y) → intero
  power ← 1
  IF y > 0 THEN
    power ← potenza(x, y/2)
    power ← power * power
    IF y è dispari THEN
      power ← power * x
  RETURN power
```

Funzione di ricorrenza associata

$$T(x, y) = \begin{cases} 3 & y = 0 \\ 7 + T(x, \lfloor \frac{y}{2} \rfloor) & \text{altrimenti} \end{cases}$$

→ $T(x, y) = 7 \cdot \log_2(y) + 10$

Stessa crescita rispetto all'algoritmo originale

Confronto tra funzioni

date due funzioni f, g quale “cresce” di più

$3n^2$

$7 \log_2(y) + 10$

$2000000n$

$6 \log_2 y + 8$

$\Theta(\log(n))$

1. Prodotti iterati → $T(x, y) = 3y + 3 = O(y)$ **crescita lineare**
2. Potenze ricorsive

a. $T(x, y) = 6 \log_2 y + 8$

→ $O(\log y)$ **crescita logaritmica**

b. $T(x, y) = 7 \log_2 y + 10$

le basi dei logaritmi sono poco rilevanti

Considerazioni sull’algoritmo per il calcolo della potenza

per i numeri reali

$$x^y = x^{\frac{y}{2} \cdot 2} = (x^{\frac{y}{2}})^2 = (x^2)^{\frac{y}{2}}$$

per i numeri interi

$$x^y = \begin{cases} (x^2)^{\frac{y}{2}} & \text{per } y \text{ pari} \\ (x^2)^{\frac{y-1}{2}} & \text{per } y \text{ dispari} \end{cases}$$

Algoritmo “alla russa” per le potenze

```
ALGORITMO potenza(intero x, intero y) → intero
  power ← 1
  WHILE y > 0 DO
    IF y è dispari THEN
      power ← power * x
      y ← y/2      // divisione intera
      x ← x * x
    RETURN power
```

- Tempo $\Theta(\log y)$
- Spazio $O(1)$

Osservazioni

$k = 5$ cifre binarie → $10000_2 = 2^4_{10}$
→ $11111_2 = 2^{5-1}$

con k cifre → $2^{k-1} \leq n \leq 2^k$

$$\begin{aligned} k - 1 &\leq \log_2 n < k \\ k &\leq \log_2 n + 1 < k + 1 \end{aligned}$$

→ $\lfloor \log_2 n \rfloor + 1$ numero di cifre binarie per scrivere n
per numeri in base 10 il discorso è analogo.

A ← B * C

Cosa rappresenta questa operazione? Potrebbe essere un prodotto tra matrici
Quali operazioni costano poco? Serve stabilire una macchina comune

 Quali operazioni *costano* tempo costante - $O(1)$

Macchina di Turing

introdotta per capire ciò che i computer possono fare
È composta da:

- Nastro di input illimitato che serve anche come memoria con accesso sequenziale
- Un programma
- Un nastro di output

Macchina ad accesso diretto (RAM)

Schema teorico più applicabile

Composto da:

- Nasto di input
- Programma
- Nastro di output
- **Memoria ad accesso diretto** potenzialmente illimitata
 - ogni cella ha una sua posizione - **indirizzo**
 - tramite **istruzioni di accesso a memoria** load/store
 - tempo di accesso **indipendente** dalla posizione - costante
- tempo costante
 - **operazioni aritmetico-logiche** + * - /
 - operazioni di confronto < > =
 - salto

Algoritmo trova minimo

```
ALGORITMO minimo(sequenza s) → elemento
  min ← primo elemento di s
  WHILE non hai ispezionato tutta s DO
    x ← prossimo elemento di s
    IF x è minore di min THEN min ← x
  RETURN min
```

supponendo `n = #elementi di s`

- $n - 1$ iterazioni del ciclo
- assegnamenti $2n - 1$ nel caso peggiore
- confronti $n - 1$

Se i tempi di assegnamenti e confronti sono costanti $O(1)$ il tempo cresce come $\Theta(n)$

In caso di **interi** questo vale.

In caso di **stringhe**? No. Dipende quanto sono lunghe le stringhe

dato $x > 0$, calcolare x^x

```
ALGORITMO xx(intero x) → intero
  p ← 1
  FOR i ← 1 TO x DO
    p ← p * x
  p
```

`#assegnamenti` $\Theta(x)$

`#confronti/incrementi` $\Theta(x)$

 Lezione 5 - @October 3, 2025

Modelli di costo e complessità

Costo uniforme

- Modello RAM con **memoria ad accesso diretto**:
 - operazioni di accesso a memoria `load/store` con tempo indipendente dalla posizione
 - operazioni aritmetico-logiche `+ - * /` a costo costante
 - operazioni di confronto `< > =` e salti a costo costante
- **Criterio di costo uniforme**:
 - *tempo*: ogni istruzione elementare costa 1 unità di tempo, indipendentemente dalla grandezza degli operandi
 - *spazio*: ogni variabile elementare occupa 1 unità di spazio, indipendentemente dal valore contenuto
- È ragionevole quando i valori hanno grandezza limitata (es. interi o floating nei range `int, long, float, double`).
- È inadeguato quando si manipolano quantità **arbitrariamente grandi**: bisogna tenere conto della dimensione della rappresentazione (numero di bit / caratteri).

Costo logaritmico

- **Criterio di costo logaritmico**:
 - il tempo di ciascuna operazione è proporzionale alla **lunghezza della rappresentazione** degli operandi
 - lo spazio misura la lunghezza della rappresentazione del dato
- Esempi:
 - interi → lunghezza = numero di bit
 - stringhe → lunghezza = numero di caratteri
- Libreria Go `math/big`: implementa interi di grandezza arbitraria, dove il costo delle operazioni dipende dalla lunghezza dei numeri.

Esempio: algoritmo `xx` per calcolare `x^x`

```
ALGORITMO xx(intero x) → intero
  p ← 1
  FOR i ← 1 TO x DO
    p ← p * x
  p
```

- Numero di assegnamenti: $\Theta(x)$
- Numero di confronti/incrementi: $\Theta(x)$
- Con **costo uniforme**: tempo $\Theta(x)$, spazio $\Theta(1)$ (poche variabili: `i, p, x`).

Effetto del costo logaritmico su `xx`

- All’**i-esima iterazione** il valore di `p` è x^i .
- Se il costo per moltiplicare due interi è proporzionale al **numero di bit** dei loro operandi, il costo dell’**i-esima iterazione** è $\Theta(i \cdot \log x)$ (perché il risultato x^i ha circa $i \cdot \log x$ bit).
- Costo complessivo:

$$\sum_{i=1}^x \Theta(i \cdot \log x) = \Theta(\log x \cdot \sum_{i=1}^x i) = \Theta(\log x \cdot x^2)$$

- Con **costo logaritmico** il tempo dell’algoritmo `xx` è quindi $\Theta(x^2 \cdot \log x)$.

Complessità di tempo di un algoritmo

- Fissato un algoritmo `A` e un’istanza di input `I`:
 - `tempo(I)` = tempo impiegato da `A` su input `I`.
- Vogliamo studiare come cresce il tempo al crescere della **lunghezza dell’input** `|I|`.
- Funzione tempo nel **caso peggiore**:

$$T(n) = \max\{tempo(I) \mid |I| = n\}$$

Complessità media (tempo medio)

- `tempo(I)` = tempo impiegato da `A` su input `I`.
- Supponiamo di avere, per ogni istanza `I` di lunghezza `n`, una probabilità `Prob(I)`.
- **Tempo medio**:

$$T_{avg}(n) = \sum_{|I|=n} Prob(I) \cdot tempo(I)$$

- Esempio (istanze equiprobabili): si fa la media aritmetica dei tempi.

Tempi polinomiali vs esponenziali

- **Algoritmi polinomiali**: tempo limitato da un polinomio nella lunghezza dell’input (`n, n^2, n^3, ...`).
 - Considerati algoritmi “ragionevoli/praticabili”.
- **Algoritmi esponenziali**: tempo `2^n, 3^n, ...` .
 - Considerati “impraticabili” per input anche moderati.

Tabella di confronto (1 miliardo di operazioni al secondo)

- `n` operazioni → tempo lineare (nano-secondi).
- `n^2, n^3` → tempi ancora gestibili anche per `n` grandi.
- `2^n` :
 - per `n = 50` → circa `10^15` operazioni (11.5 giorni).
 - per `n = 60` → `10^18` operazioni (~32 anni).
 - per `n = 100` → `10^30` operazioni (~3·10^10 millenni!).

Investire in hardware o in algoritmi?

- Computer sempre più veloci: se aumento la velocità di un fattore `k`, di quanto cresce la dimensione delle istanze risolvibili nello stesso tempo `t` ?
- Per un algoritmo con tempo:

tempo	computer attuale	computer 100 volte più veloce	computer 1000 volte più veloce
n	N_1	$100N_1$	$1000N_1$
n^2	N_2	$10N_2$	$31.6N_2$
n^3	N_3	$4.64N_3$	$10N_3$
2^3	N_4	$N_4 + 6.64$	$N_4 + 9.97$

- Conclusione: per algoritmi esponenziali l’hardware più veloce porta **benefici limitatissimi**.

Complessità di problemi

- Fissato un problema $\mathcal{P}$ e la risorsa *tempo*.

- **Upper bound** (limite superiore):
 - Se esiste un algoritmo $\mathcal{A}$ che risolve $\mathcal{P}$ in tempo $T(n)$, allora $\mathcal{P}$ è **risolubile in tempo** $O(T(n))$.
- **Lower bound** (limite inferiore):
 - Se si dimostra che **ogni** algoritmo che risolve $\mathcal{P}$ deve usare almeno tempo $T'(n)$, allora $\mathcal{P}$ **richiede tempo** $\Omega(T'(n))$.
- Definizioni analoghe valgono per altre risorse (es. spazio).